

1. Which list implementation did you use as the snake's body?

I decided to use linked lists as the snake body.

2. Why did you select that implementation?

Honestly, I just randomly chose it. It allowed me to test if my Linked List was implemented properly (it wasn't).

3. Which list operations occur during a normal snake movement?

The operations that occur is `get()`, `.row` or `.column`, `getlength()`, `insert()`, `remove()`, and `spawnfood()`

4. Which operations occur when the snake eats food?

`spawnfood()`, which calls `getcustomrandom()`, `getlength()`, and `get()`.

5. What is the worst-case runtime for one movement step?

n^3 , as draw had 3 nested for loops.

6. How does collision detection affect the runtime of a game step?

It affects by $o(n)$ and $O(N^2)$. The main causes is an if statement with four logic checks ($O(N)$), and a for loop ($O(n)$).

7. What advantages does the array-based implementation provide?

It is much easier to logically implement array based lists, as they use less pointers and nodes.

8. What advantages does the linked implementation provide?

It is much faster, and can do more complicated code relatively easier.

9. Which implementation would you choose if indexed access were the most common Operation?

Array based lists. Indexing is way easier on array based lists,so it is best to use that.

10. Which implementation would you choose if insertion and removal at the front were the most common operations?

I would choose linked lists. In the context of the snake game, both are pretty straightforward, but in more complicated cases i would prefer linked lists.